

WEEK-9

Task-1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

```
GNU nano 8.7.1                                jobtable.c *
#include <stdio.h>
#include <unistd.h>

struct Job {
    int id;
    int pid;
    char command[50];
    char status[20];
};

int main() {
    struct Job jobs[] = {
        {1, 1234, "sleep 30", "Running"},
        {2, 1235, "sleep 50", "Stopped"}
    };

    int n = 2;

    printf("Job ID\tPID\tStatus\t\tCommand\n");
    printf("-----\n");

    for (int i = 0; i < n; i++) {
        printf("[%d]\t%d\t%-10s\t%s\n",
            jobs[i].id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
    }

    return 0;
}|

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

Output:

```
root@Mani:~/skill$$ cd skill
root@Mani:~/skill$$ nano jobtable.c
root@Mani:~/skill$$ gcc jobtable.c -o jobtable
root@Mani:~/skill$$ ./jobtable
Job ID  PID      Status      Command
-----
[1]     1234     Running     sleep 30
[2]     1235     Stopped     sleep 50
root@Mani:~/skill$$ |
```

Report: In this task, I implemented a simple job table to store and display active jobs. Each job was maintained with its job ID, process ID, command, and current status. I tested the listing with different job states such as Running and Stopped. This helped me understand how a shell keeps track of background processes.

Task-2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

Code:

```
GNU nano 8.7.1 foreground.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process running...\n");
        sleep(5);
        printf("Child completed.\n");
    }
    else {
        printf("Foreground job PID: %d\n", pid);
        waitpid(pid, NULL, 0);
        printf("Job completed. Shell gets control back.\n");
    }

    return 0;
}
```

Output:

```
root@Mani:~/skill$$ nano foreground.c
root@Mani:~/skill$$ gcc foreground.c -o foreground
root@Mani:~/skill$$ ./foreground
Foreground job PID: 1196
Child process running...
Child completed.
Job completed. Shell gets control back.
root@Mani:~/skill$$ |
```

Report: In this task, I implemented foreground process handling using `fork()` and `waitpid()`. The parent process waited for the child process to complete before continuing. This demonstrated how a shell gives control to a foreground job and gets control back after execution. The program was tested successfully with a child process.

Task-3: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

Code:

```

GNU nano 8.7.1                               signals.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received\n");
    else if (sig == SIGTSTP)
        printf("\nSIGTSTP received\n");
    else if (sig == SIGCONT)
        printf("\nSIGCONT received\n");

    fflush(stdout);
}

int main() {
    signal(SIGINT, handler);
    signal(SIGTSTP, handler);
    signal(SIGCONT, handler);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C or Ctrl+Z to test signals.\n");

    while (1) {
        printf("Running...\n");
        sleep(3);
    }

    return 0;
}

```

Output:

```

Job completed. Shell gets control back.
root@Mani:~/skill$$ nano signals.c
root@Mani:~/skill$$ gcc signals.c -o signals
root@Mani:~/skill$$ ./signals
PID: 1347
Press Ctrl+C or Ctrl+Z to test signals.
Running...
Running...
Running...
Running...
Running...
^C
SIGINT received
Running...
Running...
Running...
^Z
SIGTSTP received
Running...
Running...
|

```

Report:

In this task, I created a program to handle signals such as SIGINT and SIGTSTP. I tested the program using **Ctrl+C** and **Ctrl+Z** and observed the corresponding signal messages. I also studied process IDs, process groups, and signal information using Linux commands. This helped me understand how the terminal communicates with running processes.

Task-4: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery

Code:

```
GNU nano 8.7.1          resume.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child running. PID = %d\n", getpid());

        while (1) {
            printf("Child working...\n");
            sleep(2);
        }

        sleep(2);

        printf("Stopping child...\n");
        kill(pid, SIGSTOP);

        sleep(2);

        printf("Child stopped. Sending SIGCONT...\n");
        kill(pid, SIGCONT);

        sleep(5);

        printf("Terminating child...\n");
        kill(pid, SIGTERM);

        waitpid(pid, NULL, 0);

        printf("Child terminated successfully.\n");

        return 0;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

Output:

```
Dash: kill: (1547)  No such process
root@Mani:~/skill$$ nano resume.c
root@Mani:~/skill$$ gcc resume.c -o resume
root@Mani:~/skill$$ ./resume
Child running. PID = 1471
Child working...
Child working...
Stopping child...
Child stopped. Sending SIGCONT...
Child working...
Child working...
Child working...
Terminating child...
Child terminated successfully.
root@Mani:~/skill$$ |
```

Report: In this task, I demonstrated how a stopped process can be resumed using the SIGCONT signal. The child process was first stopped using SIGSTOP and then continued using SIGCONT. Its execution was monitored until it was finally terminated. This experiment helped me understand how shells recover and manage stopped background jobs.